

Prog 4 – ZH-2

Bevezetés

A bevezetést és a feladatokat figyelmesen olvassa végig.

A ZH-t zip fájlként kell feltölteni az alábbi formátumban: `név_neptun_kód.zip` (tartalmazza a zh2-feladat és a zh2-ws-feladat projektet is)

Az alkalmazáserver helye a labor gépein: `C:\apache-tomcat`

Nem forduló kód vagy el nem induló alkalmazás automatikusan 0 pontos ZH-t eredményez.

ChatGPT (vagy bármilyen AI) használata, illetve egymás segítése tilos, ennek észrevétele (akár a ZH során, akár a beadott kód alapján) szintén 0 pontos ZH-t eredményez.

Órai anyag és internet használható a feladatok megoldása során, viszont teljes blokkok másolása esetén az adott feladat 0 pontot ér.

A 0. feladat git kezeléshez fog kapcsolódni, az ott leírtak beugrónak számítanak.

Git hiánya esetén a ZH 0 pontos.

A teljes ZH egyben történő commitálása esetén a ZH szintén 0 pontos.

Több feladat egyben történő commitálása esetén csak az egyik feladat lesz figyelembe véve a javítás során (figyeljen arra, hogy egy feladatot egyben commitálja).

Mivel a ZH jelentős része kapcsolódik az adatbázishoz és az adatok is ott kerülnek tárolásra, így a lentebb megadott Resource segítségével működő adatbázis kapcsolat hiánya esetén a ZH 0 pontot ér.

Ha a docker container létrehozása esetén hibaüzenetet kapna, akkor a docker desktop-on jobb egérgombot nyomva válassza a „Switch to linux containers” lehetőséget, majd a docker beállításai között válassza a „Docker Engine” menüpontot és konfigurációként állítsa be a következőt:

```
{  
  "builder": {  
    "gc": {  
      "defaultKeepStorage": "20GB",  
      "enabled": true  
    }  
  },  
  "experimental": true  
}
```

Ellenőrizze, hogy a `mariadb-java-client-3.1.3.jar` megtalálható-e a tomcat alatt, ha nem akkor a gyakorlaton mutatott módon töltsse le az internetről és helyezze el a megfelelő könyvtárban.

A feladathoz szükség van egy adatbázisra. Amennyiben elindítja a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbázis kapcsolathoz szükséges resource definíciója (ha már van másik resource felvéve azt törölje ki):

```
<Resource name="jdbc/ZH2MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/zh2_database" username="zh2_user"
password="ZH2Password111" maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban két felhasználó van.

user1: neki user jogosultsága van

user2: neki user és creator jogosultsága van

Mind a kettőnek "Password111" a jelszava. A jelszavak Bcrypttel vannak elforgatva.

Git (zh2-feladat és zh2-ws-feladat)

- Hozzon létre egy git repository-t a kapott projektekben.
- Készítsen egy develop branchet, majd ezen a branchen dolgozzon.
- A kapott projekt fájlokat commitálja el egyben.
- Minden feladat megoldását külön-külön commitálja el, figyeljen a beszédes commit message-kre.

Adatbázis

1. Implementálja a következő adat elérési osztályokat és a bennük lévő metódusokat:

Repository, FoodRepository, RoleRepository, UserRepository

SOAP

2. A zh2-ws-feladat projektben található FoodDataService és SoapFoodDataService osztályok hiányzó részeit egészítse ki, majd a két osztály alapján generáljon WSDL-t az src/main/resources/wsdm mappába (cxf-java2ws-plugin).

Az alkalmazás és a web service a 8081-es porton legyen elérhető.

(Az „IntelliJ Idea” Tomcat konfiguráció „Server” tab-ján a „Tomcat Server Settings” alatt található JMX portot állítsa át 1098-ra.)

3. A kigenerált WSDL-eket másolja át a zh2-feladat projekt src/main/resources/wsdm mappába és generáljon a zh2-feladat projektben egy klienst ehhez a web service-hez. Ezeket az osztályokat közvetlenül a target/generated mappába generáltassa ki (jaxws-maven-plugin).

Az alkalmazás a 8080-as porton legyen elérhető.

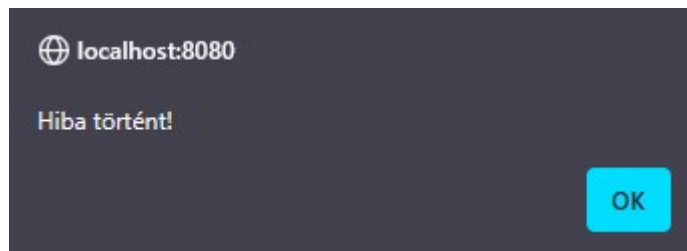
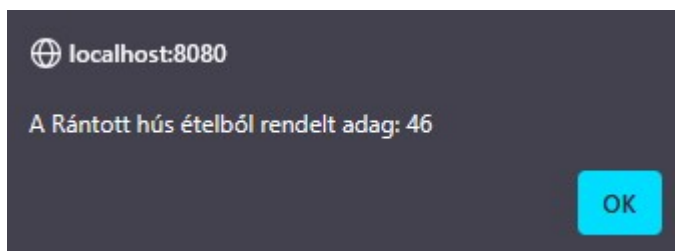
4. A FoodService osztálynak implementálja a getFoodPortion metódusát. A kigenerált WS klienst használva hívja meg a zh2-ws-feladat projektben elkészített web service-t.

REST

5. Implementálja a FoodController osztályt, mint REST controller osztály. Várjon JSON-öket befelé jövet és JSON-öket is adjon kifelé is.

Hozzon benne létre methodokat, amik meghívják a FoodService osztályban található funkciókat. Figyeljen a path-ekre.

6. A foodList.jsp oldalon található „Rendelt adag” gombot implementálja. Hívja meg JavaScript segítségével a fentebb implementált REST endpointok közül azt, amelyik a WS-t hívja. Akár sikeres volt a hívás, akár nem, a böngésző dobjon róla egy értesítést.



Security (zh2-feladat projekt)

7. A login.html állományba készítse el a bejelentkező formot.

8. Implementálja az AuthenticationModule osztálynak a metódusait úgy, hogy az adatbázisból authenticáljon a rendszer. Ehhez használja a UserRepository és a RoleRepository osztályokat.

9. Konfigurálja be az authenticálást a különböző állományokban a következőképpen:

- Legyen FORM alapú bejelentkezés.
- Az index.jsp-t el lehessen érni bejelentkezés nélkül.
- Az összes többi oldal eléréséhez be kell jelentkezni és a “user” role-hoz legyen kötve.
- Az új étel felvétele a “creator” role-hoz legyen kötve.
- A bejelentkező oldal legyen a login.html.
- Hibás bejelentkezés esetén irányítson át a login_failed.html-re.

10. Implementálja a kijelentkezést is a /logout pathra. Ha sikeres volt a kijelentkezés irányítsa át a rendszer a felhasználót a context path-re.